Cletus: Section 1, Day 2

> To copy code: select it here, or click a "Copy on GitHub" link and use the copy button there. Full note online: https://github.com/mangucletus/AI-Engineering-Course/blob/main/section-1/cletus-section1-day2.md

- Date: [YYYY-MM-DD]
- Course: Week 1 Day 2, the building blocks of LLM engineering, frontier vs open-source models, and the Chat Completions API
- Lab notebook: week1/day2.ipynb (solution in week1/solution.py)
- Time spent: [e.g. 2h]

Key concepts

- The course is built around three things: models (closed frontier ones and open-source ones), tools (Ollama, Hugging Face, Gradio and so on) and techniques (APIs, prompting, RAG, fine-tuning, agents).
- Frontier models: GPT from OpenAI, Claude from Anthropic, Gemini from Google and Grok from xAI. They're the strongest, and you pay per token.
- Open-source (open-weight) models: Llama from Meta, Mistral, DeepSeek, Qwen and Gemma. Free to download, and the small ones run locally through Ollama.
- Distillation: training a small model to copy a bigger one. deepseek-r1:1.5b is DeepSeek R1 distilled into a small Qwen model.
- Chat Completions API: you give it a conversation and it predicts the next message. OpenAI came up with it and now almost every provider supports it.
- Endpoint: a URL that we send an HTTP request to. The openai Python package is only a small wrapper around that request. It doesn't contain any model.
- OpenAI-compatible endpoints: Gemini, Ollama and others accept the same request format, so the same OpenAI client works for all of them. Only base_url and api_key change.

Code / examples

Python

The Python code runs the same on Linux and Windows. Run these in notebook cells inside the week1 folder.

The same call done two ways:

Copy on GitHub

```
import os
import requests
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv(override=True)

# 1) Raw HTTP request to the endpoint
headers = {
    "Authorization": f"Bearer {os.getenv('OPENAI_API_KEY')}",
    "Content-Type": "application/json",
}
payload = {
    "model": "gpt-5-nano",
    "messages": [{"role": "user", "content": "Tell me a fun fact"}],
}
r = requests.post(
    "https://api.openai.com/v1/chat/completions", headers=headers, json=payload
)
print(r.json()["choices"][0]["message"]["content"])

# 2) Same thing with the client library
openai = OpenAI()
response = openai.chat.completions.create(model="gpt-5-nano", messages=payload["messages"])
print(response.choices[0].message.content)
```

Same client pointed at Ollama. Only base_url and the key change:

Copy on GitHub

```
# Any text works as the key for a local Ollama
ollama = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
messages = [{"role": "user", "content": "Tell me a fun fact"}]
response = ollama.chat.completions.create(model="llama3.2", messages=messages)
print(response.choices[0].message.content)
```

Optional, Gemini (needs GOOGLE_API_KEY in .env):

Copy on GitHub

```
GEMINI_BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
gemini = OpenAI(base_url=GEMINI_BASE_URL, api_key=os.getenv("GOOGLE_API_KEY"))
messages = [{"role": "user", "content": "Tell me a fun fact"}]
response = gemini.chat.completions.create(model="gemini-3.1-flash-lite", messages=messages)
print(response.choices[0].message.content)
```

Ollama commands

Check Ollama is running. It should print "Ollama is running".

Linux (Ubuntu):

Copy on GitHub

```
curl http://localhost:11434
```

Windows (PowerShell):

Copy on GitHub

```
curl.exe http://localhost:11434
```

If it isn't running, start it.

Linux (Ubuntu):

Copy on GitHub

```
sudo systemctl start ollama
```

Windows (PowerShell), or open the Ollama app from the Start menu:

Copy on GitHub

```
ollama serve
```

Download the models used today (same on both):

Copy on GitHub

```
ollama pull llama3.2
ollama pull deepseek-r1:1.5b
```

Adding a Gemini key (optional)

Get a free key at https://aistudio.google.com/api-keys, then add it to .env in the llm_engineering folder.

Linux (Ubuntu):

Copy on GitHub

```
echo "GOOGLE_API_KEY=your-gemini-key-here" >> .env
```

Windows (PowerShell):

Copy on GitHub

```
Add-Content .env "GOOGLE_API_KEY=your-gemini-key-here"
```

Exercise / challenge

Homework: change the Day 1 summarizer to use Llama 3.2 through Ollama instead of OpenAI. Only two things change, the client (base_url) and the model name. - Compare Llama's summary with GPT's for the same site. - Running locally costs nothing and the data stays on your machine, but the model is much weaker than the frontier ones.

To try the course's solution, run this from the llm_engineering folder (same on both). It asks for a URL and prints the summary.

Copy on GitHub

```
cd week1
uv run solution.py
```

Questions & blockers

Things to watch out for: - llama3.3 and llama4 are too big for a normal laptop. Stick with llama3.2, or llama3.2:1b if it's slow. - After editing .env, save it and reload with load_dotenv(override=True). - Gemini keys start with AIz (or AQ.).

To discuss with my partner

- When would we pick a local open-source model over a frontier API (cost, privacy, quality)?
- How different were the Llama and GPT summaries?

Resources

- Endpoints and HTTP basics: guides/04_technical_foundations.ipynb
- Code for Ollama, Gemini and OpenRouter: guides/09_ai_apis_and_ollama.ipynb
- Ollama: https://ollama.com